

Week6

Task1:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

char history[MAX_COMMANDS][MAX_LENGTH];
int count = 0;

void storeCommand(char command[]) {
    if (count < MAX_COMMANDS) {
        strcpy(history[count], command);
        count++;
    } else {
        for (int i = 0; i < MAX_COMMANDS - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }
        strcpy(history[MAX_COMMANDS - 1], command);
    }
}

void displayHistory() {
    if (count == 0) {
        printf("History is empty.\n");
        return;
    }
    printf("\nCommand History:\n");
    for (int i = 0; i < count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }
}

void retrieveCommand(int number) {
    if (number < 1 || number > count) {
        printf("Invalid history number.\n");
        return;
    }
    printf("Command: %s\n", history[number - 1]);
}
```

```

int main() {
    char command[MAX_LENGTH];
    int choice, number;

    while (1) {
        printf("\n--- COMMAND HISTORY ---\n");
        printf("1. Store Command\n");
        printf("2. Display History\n");
        printf("3. Retrieve Command\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice) {
            case 1:
                printf("Enter command: ");
                fgets(command, MAX_LENGTH, stdin);
                command[strcspn(command, "\n")] = '\0';

                storeCommand(command);
                printf("Command stored.\n");
                break;

            case 2:
                displayHistory();
                break;

            case 3:
                printf("Enter history number: ");
                scanf("%d", &number);

                retrieveCommand(number);
                break;

            case 4:
                exit(0);

            default:
                printf("Invalid choice.\n");

```

```

        }
    }

    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./new

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 1
Enter command: date
Command stored.

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice:

```

Task2:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500

typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

typedef struct {
    Command commands[MAX_COMMANDS];
    int count;
} Pipeline;

void parseCommand(char *input, Command *cmd) {
    char *token;
    cmd->argc = 0;

    token = strtok(input, " \\t\\n");

    while (token != NULL && cmd->argc < MAX_ARGS - 1) {
        cmd->args[cmd->argc] = token;
        cmd->argc++;
        token = strtok(NULL, " \\t\\n");
    }

    cmd->args[cmd->argc] = NULL;
}

int validatePipeline(Pipeline *p) {
    if (p->count < 1 || p->count > MAX_COMMANDS)
        return 0;

    for (int i = 0; i < p->count; i++) {
```

```

        if (p->commands[i].argc == 0)
            return 0;
    }

    return 1;
}

void executePipeline(Pipeline *p) {
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    for (int i = 0; i < p->count - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            exit(1);
        }
    }

    for (int i = 0; i < p->count; i++) {
        pids[i] = fork();

        if (pids[i] < 0) {
            perror("fork");
            exit(1);
        }

        if (pids[i] == 0) {
            if (i > 0) {
                dup2(pipes[i - 1][0], STDIN_FILENO);
            }

            if (i < p->count - 1) {
                dup2(pipes[i][1], STDOUT_FILENO);
            }

            for (int j = 0; j < p->count - 1; j++) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

```

```

        }
        execvp(p->commands[i].args[0],
            p->commands[i].args);
        perror("execvp");
        exit(1);
    }
}
for (int i = 0; i < p->count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}
for (int i = 0; i < p->count; i++) {
    waitpid(pids[i], NULL, 0);
}
}

int main() {
    Pipeline pipeline;
    char input[MAX_INPUT];
    char *start;
    char *end;
    pipeline.count = 0;
    printf("Enter pipeline commands separated by |:\n");
    printf("Example: ls -l | grep txt | wc -l\n");
    printf("Pipeline: ");
    fgets(input, sizeof(input), stdin);
    start = input;
    while (*start != '\0' && pipeline.count < MAX_COMMANDS) {
        end = strchr(start, '|');
        if (end != NULL) {
            *end = '\0';
        }
        parseCommand(start, &pipeline.commands[pipeline.count]);
        pipeline.count++;
        if (end == NULL) {
            break;
        }
        start = end + 1;
    }
    if (!validatePipeline(&pipeline)) {
        printf("Invalid pipeline layout.\n");
    }
}

```

```

    }
    if (!validatePipeline(&pipeline)) {
        printf("Invalid pipeline layout.\n");
        return 1;
    }

    printf("\nPipeline Structure:\n");

    for (int i = 0; i < pipeline.count; i++) {
        printf("%d. ", i + 1);

        for (int j = 0; j < pipeline.commands[i].argc; j++) {
            printf("%s ", pipeline.commands[i].args[j]);
        }

        printf("\n");
    }

    printf("\nExecution Order:\n");

    for (int i = 0; i < pipeline.count; i++) {
        printf("%d -> %s\n",
            i + 1,
            pipeline.commands[i].args[0]);
    }

    printf("\nExecuting pipeline...\n");

    executePipeline(&pipeline);

    printf("\nPipeline execution completed successfully.\n");

    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./new2
Enter pipeline commands separated by |:
Example: ls -l | grep txt | wc -l
Pipeline: ls -l | wc -l

Pipeline Structure:
1. ls -l
2. wc -l

Execution Order:
1 -> ls
2 -> wc

Executing pipeline...
111

Pipeline execution completed successfully.

```

Task3:

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");

    pid1 = fork();

    if (pid1 == -1) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        close(pipefd[0]);

        if (dup2(pipefd[1], STDOUT_FILENO) == -1) {
            perror("dup2");
            exit(EXIT_FAILURE);
        }

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp");
        exit(EXIT_FAILURE);
    }
    pid2 = fork();
    if (pid2 == -1) {

```

```

    pid2 = fork();
    if (pid2 == -1) {
        perror("fork");
        return 1;
    }
    if (pid2 == 0) {
        close(pipefd[1]);

        if (dup2(pipefd[0], STDIN_FILENO) == -1) {
            perror("dup2");
            exit(EXIT_FAILURE);
        }
        close(pipefd[0]);
        execlp("wc", "wc", "-l", NULL);
        perror("execlp");
        exit(EXIT_FAILURE);
    }
    close(pipefd[0]);
    close(pipefd[1]);
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);
    printf("Data flow completed successfully.\n");

    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./new3
Pipe created successfully.
111
Data flow completed successfully.

```

Task4:

To Support Multiple Pipes, Manage Descriptor Arrays,
Launch Multiple Processes, Coordinate Execution, Cleanup
Resources, Test Long Pipelines

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_INPUT 500
typedef struct {
    char *args[MAX_ARGS];
    int argc;
} Command;

void parseCommand(char *input, Command *cmd) {
    char *token;
    cmd->argc = 0;
    token = strtok(input, " \\t\\n");
    while (token != NULL && cmd->argc < MAX_ARGS - 1) {
        cmd->args[cmd->argc++] = token;
        token = strtok(NULL, " \\t\\n");
    }
    cmd->args[cmd->argc] = NULL;
}

int main() {
    char input[MAX_INPUT];
    Command commands[MAX_COMMANDS];
    int commandCount = 0;
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];
    printf("Enter a pipeline:\\n");
    printf("Example: ls | grep txt | sort | uniq | wc -l\\n\\n");
    printf("Pipeline: ");
    fgets(input, MAX_INPUT, stdin);
    char *start = input;
    char *end;
    while (*start != '\\0' && commandCount < MAX_COMMANDS) {
        end = strchr(start, '|');
        if (end != NULL) {
            *end = '\\0';
        }
        parseCommand(start, &commands[commandCount]);
        if (commands[commandCount].argc == 0) {
            printf("Invalid command.\\n");
            return 1;
        }
        commandCount++;
        if (end == NULL) {
            break;
        }
        start = end + 1;
    }
    if (commandCount < 2) {
        printf("At least two commands are required.\\n");
        return 1;
    }
    printf("\\nNumber of commands: %d\\n", commandCount);
    printf("Number of pipes: %d\\n\\n", commandCount - 1);
    for (int i = 0; i < commandCount - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            return 1;
        }
    }
    for (int i = 0; i < commandCount; i++) {
        pids[i] = fork();
        if (pids[i] == -1) {
            perror("fork");
            return 1;
        }
        if (pids[i] == 0) {
            if (i > 0) {
                if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1) {
                    perror("dup2 input");
                    exit(1);
                }
            }
            if (i < commandCount - 1) {
                if (dup2(pipes[i][1], STDOUT_FILENO) == -1) {
                    perror("dup2 output");
                    exit(1);
                }
            }
            if (i > 0) {
                close(pipes[i - 1][1]);
            }
            if (i < commandCount - 1) {
                close(pipes[i][0]);
            }
            commands[i].argv[0] = strtok(commands[i].args[0], " ");
            while (strtok(NULL, " ") != NULL) {
                // ... (rest of the code)
            }
            execvp(commands[i].argv[0], commands[i].argv);
            exit(1);
        }
    }
    for (int i = 0; i < commandCount; i++) {
        waitpid(pids[i], NULL, 0);
    }
    return 0;
}

```

```

        parseCommand(start, &commands[commandCount]);
        if (commands[commandCount].argc == 0) {
            printf("Invalid command.\\n");
            return 1;
        }
        commandCount++;
        if (end == NULL) {
            break;
        }
        start = end + 1;
    }
    if (commandCount < 2) {
        printf("At least two commands are required.\\n");
        return 1;
    }
    printf("\\nNumber of commands: %d\\n", commandCount);
    printf("Number of pipes: %d\\n\\n", commandCount - 1);
    for (int i = 0; i < commandCount - 1; i++) {
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
            return 1;
        }
    }
    for (int i = 0; i < commandCount; i++) {
        pids[i] = fork();
        if (pids[i] == -1) {
            perror("fork");
            return 1;
        }
        if (pids[i] == 0) {
            if (i > 0) {
                if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1) {
                    perror("dup2 input");
                    exit(1);
                }
            }
            if (i < commandCount - 1) {
                if (dup2(pipes[i][1], STDOUT_FILENO) == -1) {
                    perror("dup2 output");
                    exit(1);
                }
            }
            if (i > 0) {
                close(pipes[i - 1][1]);
            }
            if (i < commandCount - 1) {
                close(pipes[i][0]);
            }
            commands[i].argv[0] = strtok(commands[i].args[0], " ");
            while (strtok(NULL, " ") != NULL) {
                // ... (rest of the code)
            }
            execvp(commands[i].argv[0], commands[i].argv);
            exit(1);
        }
    }
    for (int i = 0; i < commandCount; i++) {
        waitpid(pids[i], NULL, 0);
    }
    return 0;
}

```

```

        if (i < commandCount - 1) {
            if (dup2(pipes[i][1], STDOUT_FILENO) == -1) {
                perror("dup2 output");
                exit(1);
            }
        }
        for (int j = 0; j < commandCount - 1; j++) {
            close(pipes[j][0]);
            close(pipes[j][1]);
        }
        execvp(commands[i].args[0], commands[i].args);
        perror("execvp");
        exit(1);
    }
}
for (int i = 0; i < commandCount - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}
for (int i = 0; i < commandCount; i++) {
    waitpid(pids[i], NULL, 0);
}
printf("\nAll pipeline processes completed.\n");
printf("All pipe descriptors cleaned up.\n");
return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ls | grep txt | sort | uniq | wc -l
5
root@DESKTOP-VMN932K:~# _

```